



ZeroNethermind

Engineering a Stateless Execution Layer in Ethereum
using Zero-Knowledge Proofs

Candidato:
Manuel Arto

Relatore:
Prof. Marco Di Felice

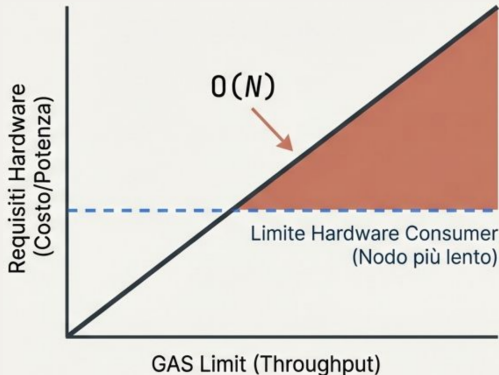
Correlatore:
Dott. Luca Sciallo

Il Trilemma e il Problema Architettonurale



Validazione simmetrica:

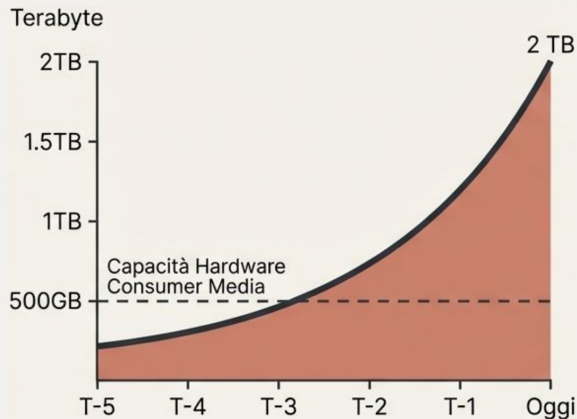
Ri-esecuzione $O(N_{\text{gas}})$



Aumentare il throughput esclude l'hardware *consumer*.

La rete è veloce quanto il suo nodo più lento.

The State Bloat Crisis



Requisiti Full Node

Storage 2TB+ NVMe SSD

IOPS > 10.000 IOPS

RAM 16GB – 32GB

Stato ~ 1.2TB Totale

Il limite è I/O-bound.

Entry barrier per i validatori.

Paradigm Shift: Zero-Knowledge Proofs

1. Completezza

Il validatore accetta sempre le computazioni corrette.

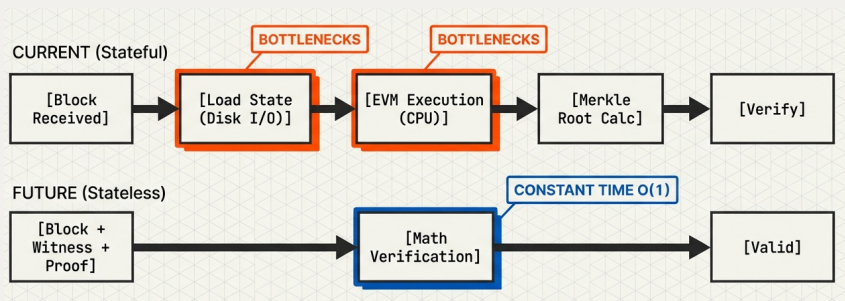
2. Solidità

Impossibilità computazionale di generare prove false.

3. Succintezza

Verifica in tempo costante
 $O(1)$ indipendente dal gas.

Da Ri-esecuzione EVM a Verifica Crittografica



Lean Ethereum: Beast vs Fort Mode



- **Beast Mode** (Datacenter centralizzati)
- **Esecuzione:** Processa transazioni (fino a 10K TPS)
- **Generazione:** Produce le prove crittografiche (Provers)



- **Fort Mode** (Hardware consumer)
- **Verifica:** Certifica l'integrità matematica del blocco
- **Sicurezza:** Nessuna ri-esecuzione, totale decentralizzazione

ZeroNethermind: L'Attester in Fort Mode

Il Paradigma "Double Zero"

Trasformazione del nodo Execution Layer in un validatore puramente *stateless*.

1. Zero State

- **Nessun DB locale:** rimosso l'albero di stato MPT.
- **Stato effimero:** solo gli header in RAM.
- **Storage azzerato:** ~ 0 GB su disco.

2. Zero Knowledge

- **Bypass EVM:** eliminata la ri-esecuzione nativa.
- **Verifica crittografica:** delegata alle prove ZK.
- **Salto prestazionale:** da $O(N_{\text{gas}})$ a $O(1)$.

ZeroNethermind: Architettura

1. Interceptor (C#)

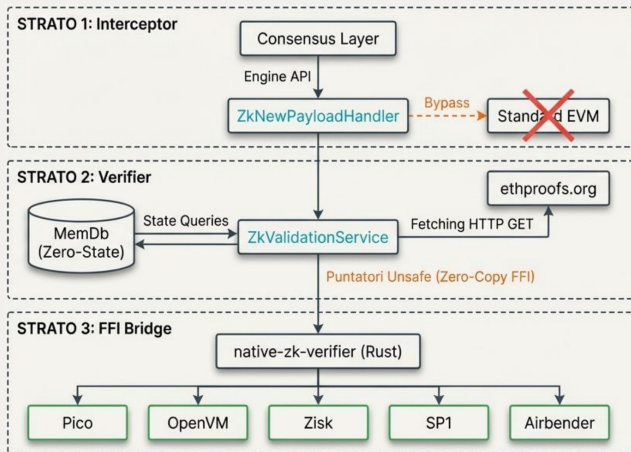
Intercetta `engine_newPayload` dal CL, bypassando la standard EVM.

2. Verifier (C#)

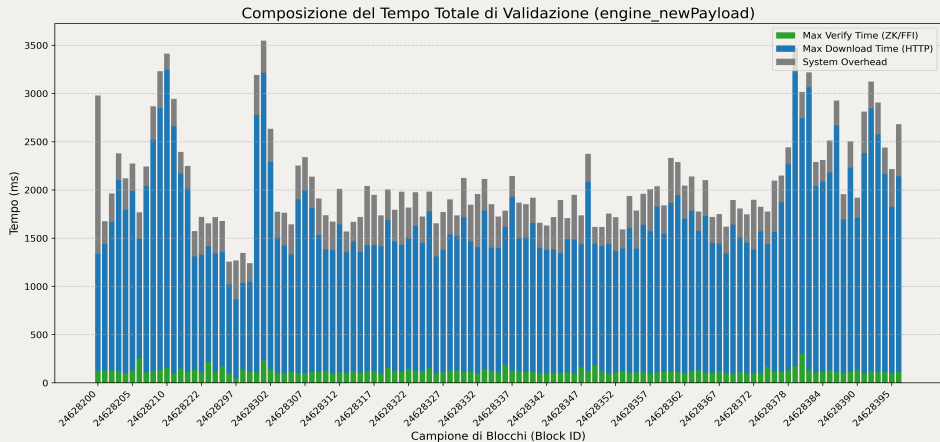
Orchestratore per il recupero delle ZKPs.

3. FFI Bridge

Interoperabilità cross-language tramite **Zero-Copy Memory**.



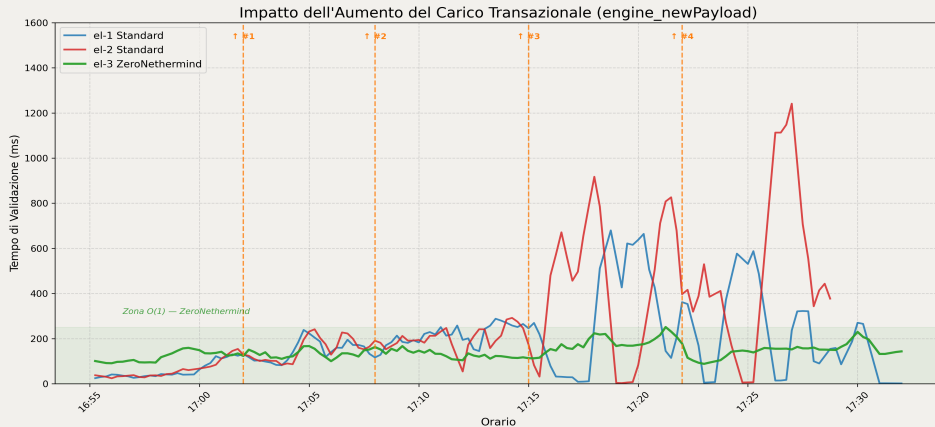
Demo 1: Validazione Live su Mainnet L1



- **Verifica FFI/Rust:** ~122 ms
- **Latenza Rete:** 94.5% del tempo

- **RAM:** Stabile < 400 MB
- **I/O Disco:** Totalmente azzerato

Demo 2: Analisi del Carico Computazionale



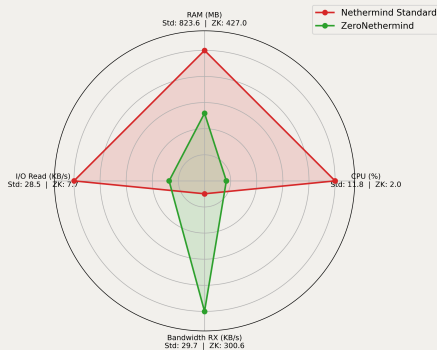
Saturazione Gas Limit
(Fino a 500M Gas)

Nodo Standard
Degradamento proporzionale.
 $\sim 57\text{ms} \rightarrow \sim 1000\text{ms}$

ZeroNethermind
Tempo costante $O(1)$.
 $\sim 128 - 147\text{ ms}$

Demo 2: Footprint Hardware

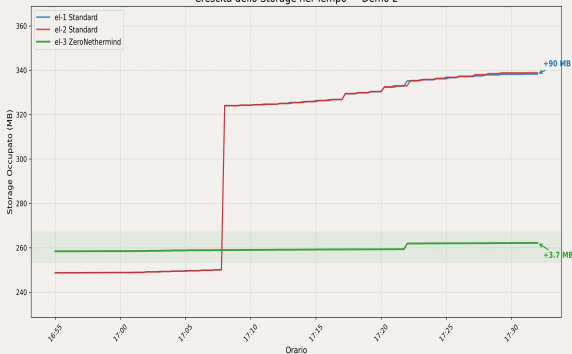
Confronto Footprint Hardware — Demo 2



Shift Computazionale

Drastica riduzione dell'overhead locale (CPU ÷6, RAM ÷2). Il carico viene interamente demandato alla Rete.

Crescita dello Storage nel Tempo — Demo 2



La crescita dello storage crolla al **4.3%** dello standard. La minaccia dello *State Bloat* è neutralizzata.

Limiti e Lavori Futuri

1. Latenza Centralizzata

Problema: Oracolo HTTP

Soluzione: EIP-8025

Propagazione delle prove ZK nativamente tramite Gossip P2P.

2. Vincoli di Timing

Problema: Sincronizzazione limitata

Soluzione: ePBS

Espansione della finestra di *proving* garantita (da 2s a 9s).

3. Data Availability

Problema: Assenza dati RPC locali

Soluzione: Portal Network

Dati storici distribuiti tramite DHT per servire i client leggeri.

Conclusioni

1. Fattibilità Dimostrata

La validazione L1 puramente *stateless* e crittografica è tecnicamente operante oggi.

2. Trilemma Risolto

Il decoupling tra esecuzione e verifica permette di scalare senza sacrificare la decentralizzazione.

3. Il Futuro

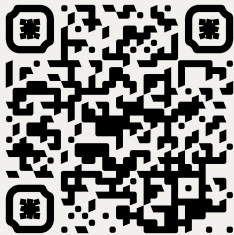
Prossimo step architetturale: nodi validatori Ethereum eseguiti nativamente su smartphone.



Grazie per l'attenzione.

Manuel Arto

manuel.arto@studio.unibo.it

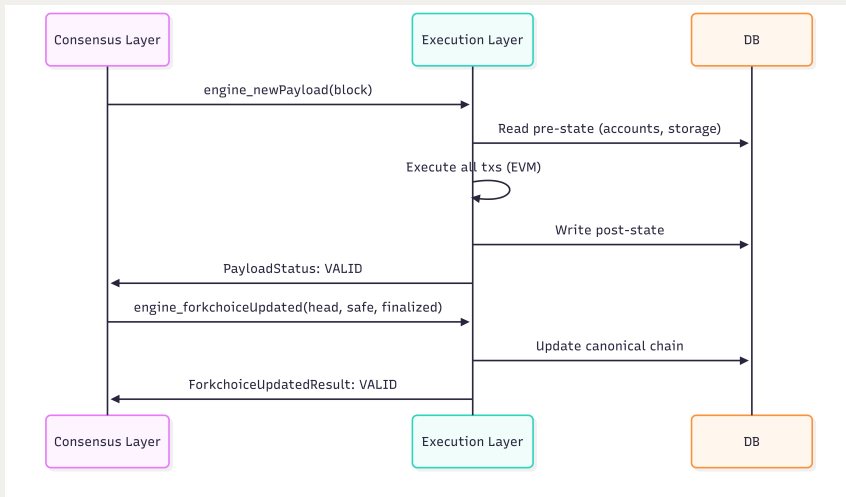


Repository ZeroNethermind

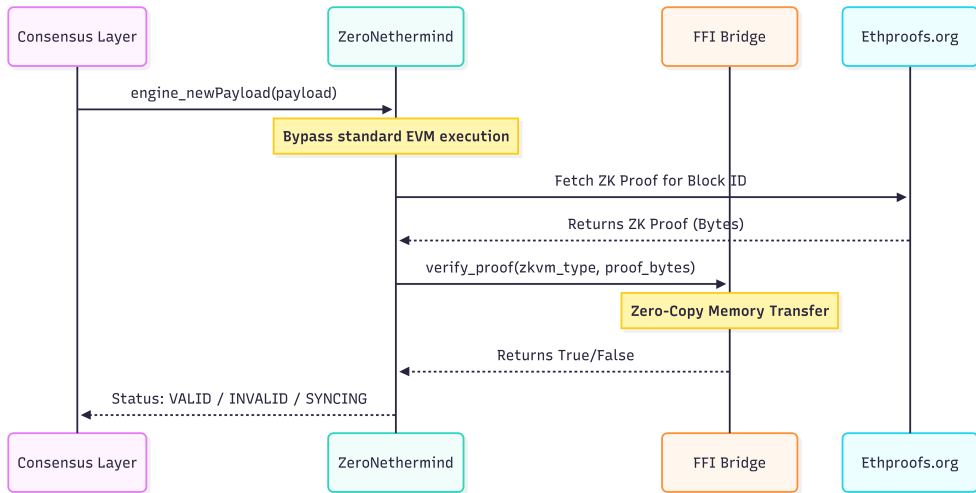
Appendice

Materiale di supporto per Q&A

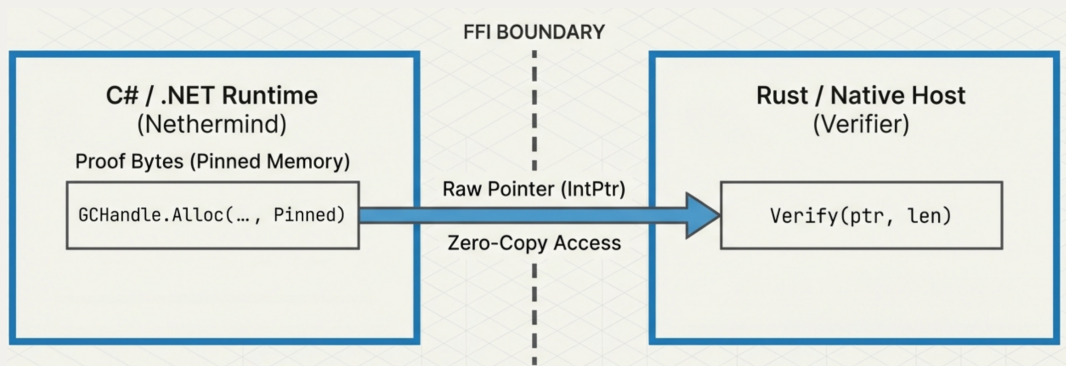
Flusso di Validazione Standard



Flusso di Validazione (ZeroNethermind)



Interoperabilità Zero-Copy: FFI Bridge



Confronti Architettureali

Beast vs Fort Mode

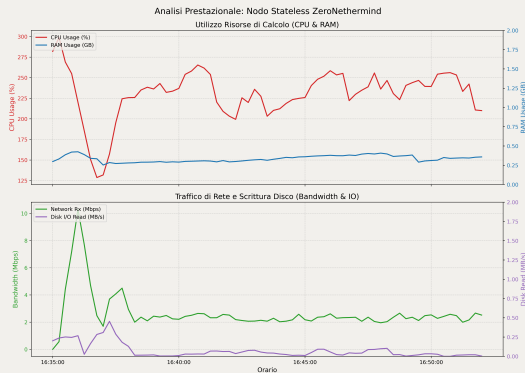
Caratteristica	Beast Mode	Fort Mode
Ruolo	Genera Prove ZK	Verifica Prove
Hardware	Specializzato	Consumer
Storage	Terabyte	~0 GB (MemDb)
Complessità	$O(N_{gas})$	$O(1)$

Standard vs ZeroNethermind

Caratteristica	Standard	ZeroNethermind
Azione	Ri-esecuzione	Verifica Locale
Risorse	Alta CPU/IO	Bassa CPU/Rete
Bottleneck	IOPS Disco	Latenza Prove
Cold Start	Ore (Snap)	Secondi

Dettagli Validazione Mainnet (Demo 1)

Risorse e Bandwidth

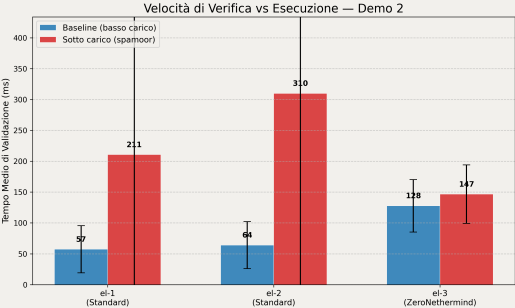


Statistiche Aggregate

Metrica	Media	Min	Max
Tempo tot. (ms)	2237	1241	9400
Max download (ms)	1768	817	8755
Max verify (ms)	122	48	301
RAM cont. (MB)	360	272	457
CPU cont. (%)	57.7	32.2	74.8
Bandwidth RX (KB/s)	333	< 1	1289
I/O read (KB/s)	77	< 1	476

Dettagli Analisi Sotto Carico (Demo 2)

Velocità di Verifica vs Esecuzione



Specifiche Hardware di Test

Componente	Specifica
OS	Manjaro Linux, kernel 6.12 (64-bit)
CPU	4x Intel Core i5-7400 @ 3.00 GHz
RAM	16 GiB DDR4
GPU	NVIDIA GeForce GTX 1060 6GB
Rete	Connessione domestica (FTTC)
Ambiente	Docker Engine con docker-compose

Sintesi del Salto Architetture

Parametro	Nethermind Full Node	ZeroNethermind
Modello Fiducia	Ri-esecuzione N-of-N	Verifica Matematica 1-of-N
Storage	~1TB (Crescita continua)	~0GB (MemDb in RAM)
RAM Richiesta	16-32 GB	< 500 MB (Plateau)
Impatto CPU	Saturazione multi-core (EVM)	Picchi transitori minimi
I/O Disco	Elevato (Accesso MPT)	Nulla
Bottleneck	Limiti Hardware (I/O-bound)	Latenza Rete (Bandwidth)
Complessità	$O(N_{gas})$	$O(1)$
Servizi Estesi	Full RPC, Block Building	Solo Attestation (Fort Mode)